Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»
КАФЕДРА ИУ7 «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

КУРСОВАЯ РАБОТА

НА ТЕМУ:

Разработка базы данных для поиска и организации досуга

Студент	ИУ7-64Б		Козин М.В.
	(группа)	(подпись, дата)	(И.О. Фамилия)
Руководитель курсовой работы			Назаренко Н.В.
		(подпись, дата)	(И.О. Фамилия)
Консультант			
		(подпись, дата)	(И.О. Фамилия)

2026 г.

РЕФЕРАТ

Расчетно-пояснительная записка 33 с., 9 рис., 14 табл., 1 прил., 16 источн.

ДОСУГ, МЕРОПРИЯТИЕ, БАЗА ДАННЫХ, РЕЛЯЦИОННАЯ МОДЕЛЬ, СИСТЕМА УПРАВЛЕНИЯ БАЗАМИ ДАННЫХ, POSTGRESQL, ТРИГГЕР.

Цель работы – спроектировать и реализовать базу данных для поиска и организации досуга и приложение для взаимодействия с ней.

В ходе работы были выделены сущности предметной области, определены требования и роли пользователей. Разработана реляционная база данных для хранения мероприятий, участников и отзывов. Реализованы приложение JoinUp, разграничение доступа и триггер записи фактов удаления объявлений. Описаны методы тестирования и приведены результаты проверки создания мероприятия, отдельных операций приложения, ограничений и прав БД, триггерной функции.

Проанализирована зависимость времени выполнения поиска от количества записей в таблице с индексацией по основным фильтрам и без нее. Для 10000 мероприятий среднее время поиска составило 4,905 мс без индекса и 3,038 мс с ним.

СОДЕРЖАНИЕ

РЕФЕРАТ **3**

ВВЕДЕНИЕ **5**

1 Аналитическая часть **6**
1.1 Анализ предметной области 6
1.2 Анализ известных решений 6
1.3 Формализация данных 7
1.4 Формализация ролей и типов 8
1.5 Анализ моделей баз данных 10
1.6 Формализация задачи 10

2 Конструкторская часть **11**
2.1 Описание сущностей базы данных 11
2.2 Ограничения целостности 13
2.3 Проверка соответствия третьей нормальной форме 14
2.4 Ролевая модель 15
2.5 Используемый триггер 16
2.6 Архитектура приложения 17
2.7 Вывод 18

3 Технологическая часть **19**
3.1 Выбор системы управления базами данных 19
3.2 Выбор средств реализации приложения 19
3.3 Структуры приложения и базы данных 19
3.4 Интерфейс приложения 20
3.5 Тестирование 22
3.6 Вывод 23

4 Исследовательская часть **24**
4.1 Технические характеристики 24
4.2 Постановка исследования 24
4.3 Определение зависимости времени поиска от количества мероприятий 25
4.4 Вывод 26

ЗАКЛЮЧЕНИЕ **27**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **28**

ПРИЛОЖЕНИЕ А. Сценарий создания объектов базы данных **29**

ВВЕДЕНИЕ

Для организации совместного досуга необходимо хранить сведения о мероприятиях, времени и месте проведения, участниках и отзывах. Размещение объявлений в общей ленте затрудняет поиск по заданным условиям и учет участников. Эти задачи решает специализированное приложение с базой данных.

Цель работы – спроектировать и реализовать базу данных для поиска и организации досуга и приложение для взаимодействия с ней.

Для достижения поставленной цели необходимо выполнить следующие задачи:

1) проанализировать предметную область и известные решения;
2) определить требования, сущности и связи между ними;
3) спроектировать БД, ограничения целостности, роли и триггер сохранения истории удаления объявлений, проверить соответствие БД третьей нормальной форме;
4) выбрать средства реализации и разработать приложение;
5) описать методы тестирования и проверить корректность основных операций;
6) исследовать время поиска мероприятий с индексом по основным фильтрам и без него.

1 Аналитическая часть

1.1 Анализ предметной области

Поиск мероприятий для совместного досуга связан с выбором тематики, места и времени проведения. Объявления размещаются в разных группах и общих лентах, поэтому пользователю приходится просматривать множество публикаций, чтобы найти подходящее мероприятие. Если дата, адрес и условия участия указаны неполно или в разных сообщениях, сравнение вариантов занимает дополнительное время.

Для организации мероприятия нужно указать его тему, время и место проведения, опубликовать описание и учитывать участников. В общей ленте объявление быстро теряется среди других записей, а изменение сведений и учет откликов затруднены. Для упрощения этих задач необходима система, в которой можно быстро найти мероприятия по интересам, городу и дате, а также разместить свое объявление и вести учет участников.

1.2 Анализ известных решений

Рассмотрим следующие существующие решения для поиска и организации мероприятий:

1) Meetup – сервис публикации и поиска встреч по интересам с учетом участников [1];
2) Mobilizon – приложение для создания мероприятий, поиска и управления участием; организатор может разрешить запись без учетной записи [2];
3) Timepad – сервис публикации и поиска мероприятий, регистрации участников и продажи билетов [3];
4) группы в социальных сетях – публикация объявлений в ленте группы с обсуждением в комментариях.

Результаты сравнения представлены в таблице 1.1. Интеграция с соцсетями означает средства публикации, регистрации или общения через соцсети и мессенджеры; универсальность – поддержку разных тематик; запись без аккаунта – участие без предварительного создания учетной записи. Фильтры поиска учитывают место, тематику и время. Для групп рассматриваются обычные публикации без отдельных сервисов мероприятий.

Таблица 1.1 — Результаты сравнения известных решений

Критерий	**Meetup**	**Mobilizon**	**Timepad**	**Группы соцсетей**	**JoinUp**
Доступность в РФ	Нет	Нет	Да	Да	Да
Интеграция с соцсетями	Нет	Нет	Да	Да	Да
Универсальность	Да	Да	Да	Нет	Да
Запись без аккаунта	Нет	Да	Нет	Нет	Нет
Фильтры поиска	Нет	Да	Да	Нет	Да

1.3 Формализация данных

БД хранит пользователей, мероприятия, локации, категории, изображения, участие, отзывы и факты удаления объявлений. Мероприятие связано с организатором и локацией, отзыв – с автором и мероприятием. Участие пользователей и принадлежность мероприятий к категориям образуют связи «многие ко многим».

На рисунке 1.1 представлена диаграмма БД «сущность-связь» в нотации Чена.

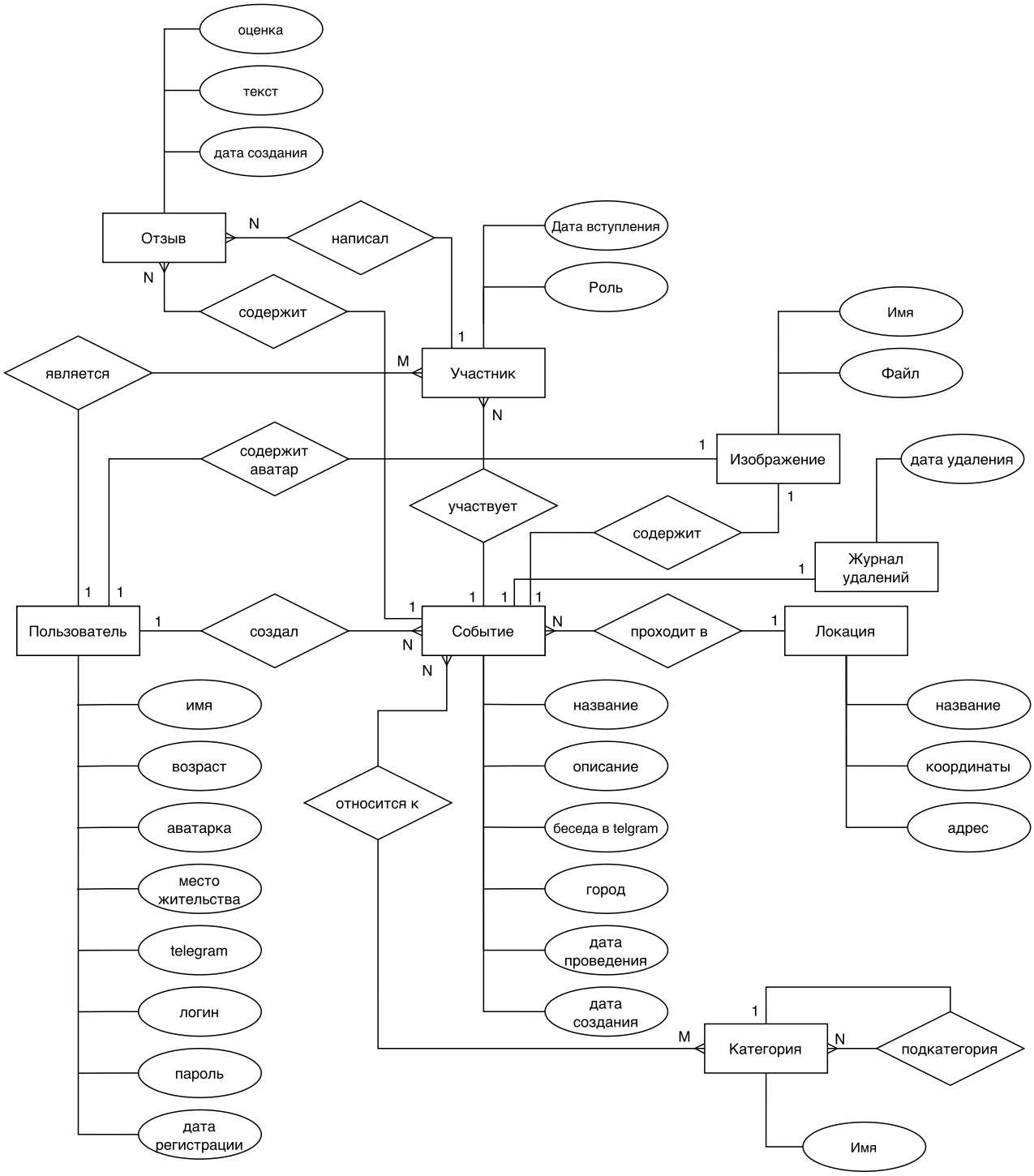


Рисунок 1.1 — Диаграмма «сущность-связь»

1.4 Формализация ролей и типов

Гость регистрируется и входит в приложение. Пользователь ищет и создает мероприятия, изменяет свои объявления, участвует в мероприятиях, оставляет отзывы и редактирует профиль. Администратор может также изменять и удалять чужие объявления. Действия показаны на рисунке 1.2.

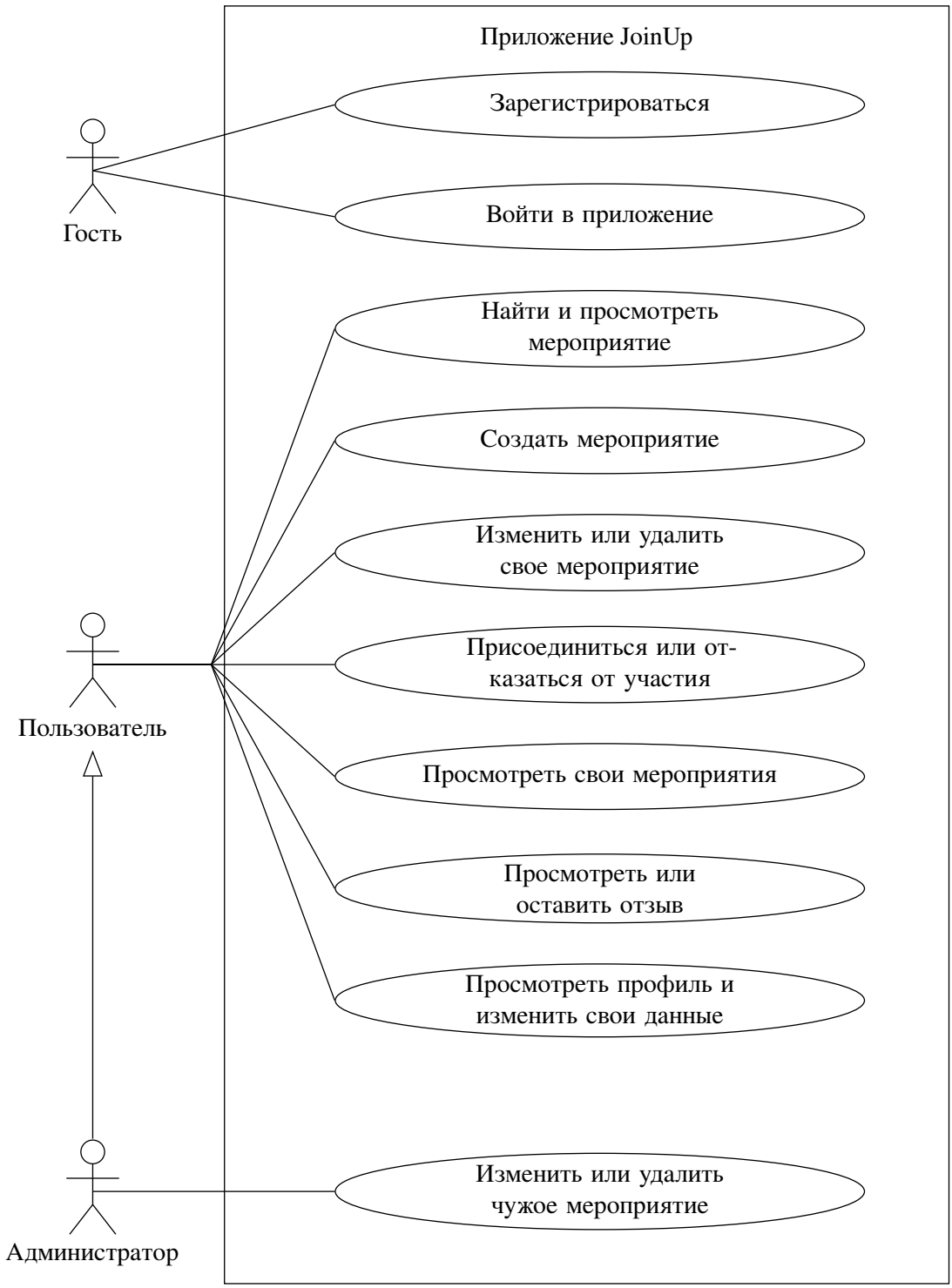


Рисунок 1.2 — Диаграмма прецедентов

1.5 Анализ моделей баз данных

Иерархическая модель представляет данные деревом: у подчиненной записи один родитель [4]. Она подходит для категорий, но связи пользователей с несколькими мероприятиями требуют дополнительных ссылок либо дублирования данных.

Документная модель хранит объекты с вложенными данными или ссылками [5]. Мероприятие можно хранить вместе с участниками, однако при копировании сведений о пользователе их изменение потребует обновления нескольких документов.

Реляционная модель представляет данные таблицами, связанными ключами [4]. Для JoinUp выбирается эта модель: состав атрибутов определен, участие и категории задаются отдельными отношениями, поиск выполняется отбором и соединением записей, рейтинг – агрегированием оценок.

1.6 Формализация задачи

Необходимо разработать реляционную БД и приложение для публикации и поиска мероприятий, учета участия и отзывов. Требуется обеспечить целостность данных, разграничение прав и запись фактов удаления объявлений триггером. После реализации необходимо описать методы тестирования, проверить корректность основных операций и исследовать зависимость среднего времени поиска от количества мероприятий с индексом по основным фильтрам и без него.

2 Конструкторская часть

2.1 Описание сущностей базы данных

На основе сущностей аналитической части определены таблицы пользователей, мероприятий, локаций, изображений, категорий, участников и отзывов. Связь мероприятий с категориями вынесена в отдельную таблицу. Факты удаления хранятся в журнале.

Структура основных таблиц приведена ниже. Все атрибуты обязательны, кроме отмеченных как необязательные. Поле `id` – первичный ключ каждой таблицы, кроме event_category, где ключ составной.

Таблица 2.1 — Структура таблицы пользователей users

Атрибут	**Тип**	**Назначение и ограничения**
id	Идентификатор	Пользователь
name, city	Строка	Имя и город проживания
age	Целое число	Возраст, больше нуля
login	Строка	Уникальный логин
password	Строка	Хеш пароля
created_at	Дата и время	Регистрация
role	Строка	Роль приложения, по умолчанию user
telegram_login	Строка	Имя в Telegram, необязательное
avatar_id	Идентификатор	Ссылка на image, необязательная

Таблица 2.2 — Структура таблицы мероприятий event

Атрибут	**Тип**	**Назначение и ограничения**
id	Идентификатор	Мероприятие
name, city	Строка	Название и город проведения
description	Строка	Описание, необязательное
created_at, updated_at, event_date	Дата и время	Создание, последнее изменение и проведение
telegram_chat_url	Строка	Ссылка на чат, необязательная
creator_id	Идентификатор	Организатор, ссылка на users
location_id	Идентификатор	Место проведения, ссылка на location
image_id	Идентификатор	Обложка, ссылка на image, необязательная
deleted	Логический	Признак удаления, изначально ложный

Таблица 2.3 — Структура таблиц локаций и изображений

Таблица	**Атрибуты**	**Тип и назначение**
location	id	Идентификатор локации
location	name, address	Строки: название и адрес
location	longitude, latitude	Вещественные числа: долгота и широта
image	id	Идентификатор изображения
image	name	Строка: имя файла
image	data	Двоичные данные: содержимое файла

Таблица 2.4 — Структура таблиц категорий и их связи с мероприятиями

Таблица	**Атрибуты**	**Тип, назначение и ограничения**
category	id	Идентификатор категории
category	name	Строка: уникальное название
category	subcategory_id	Идентификатор: необязательная ссылка на category
event_category	event_id, category_id	Идентификаторы мероприятия и категории; внешние ключи и составной первичный ключ

Таблица 2.5 — Структура таблиц участия, отзывов и журнала

Таблица	**Атрибуты**	**Тип и назначение**
member	id	Идентификатор участия
member	user_id, event_id	Идентификаторы пользователя и мероприятия; внешние ключи
member	role	Строка: роль участника мероприятия
review	id	Идентификатор отзыва
review	event_id, author_id	Идентификаторы мероприятия и автора; внешние ключи
review	rating, text	Целое число: оценка; строка: текст отзыва
review	created_at	Дата и время создания
event_deletion_log	id	Идентификатор записи журнала
event_deletion_log	event_id	Идентификатор мероприятия; внешний ключ
event_deletion_log	deleted_at	Дата и время удаления

Диаграмма БД представлена на рисунке 2.1. В таблицах указаны атрибуты и их типы; обозначение NN соответствует обязательным полям. Линии показывают связи по внешним ключам с кратностью «многие к одному». Для необязательных внешних ключей связанная запись может отсутствовать.

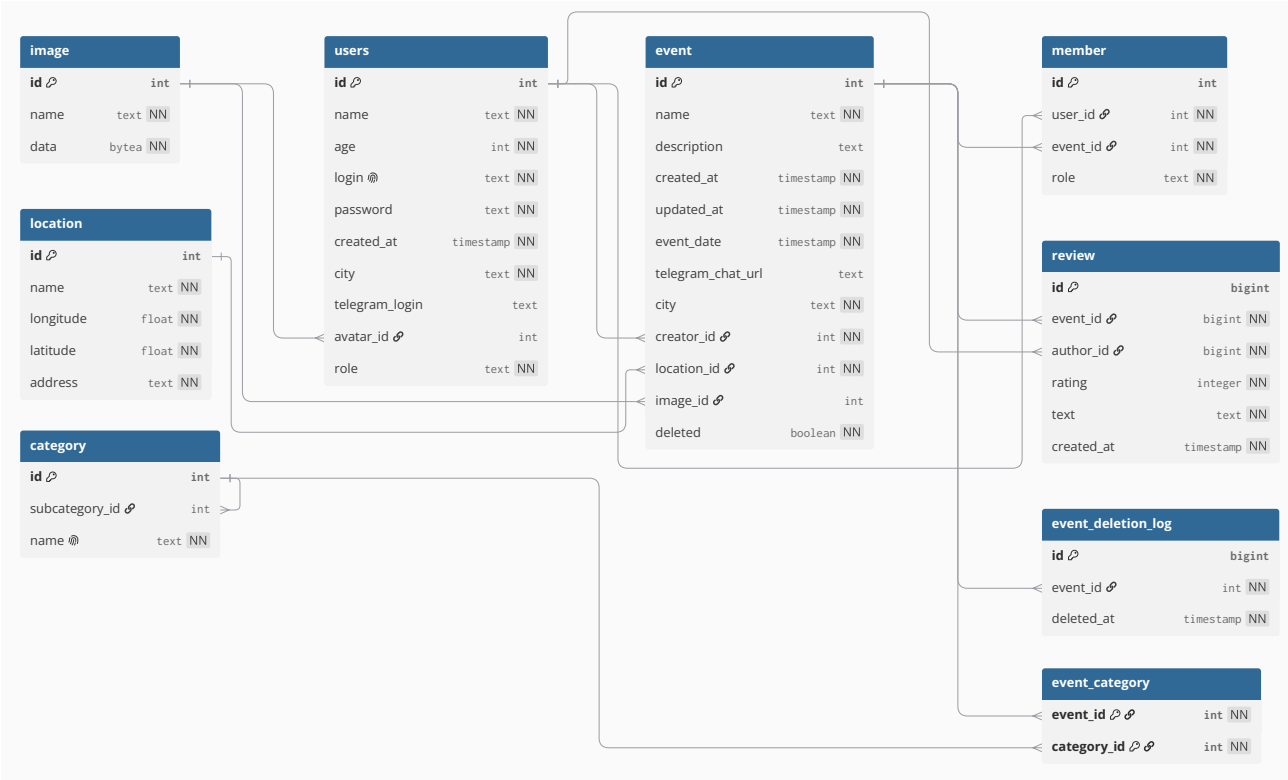


Рисунок 2.1 — Диаграмма базы данных

2.2 Ограничения целостности

Первичные ключи определяют записи, внешние ключи запрещают ссылки на отсутствующие объекты. Дополнительные ограничения приведены в таблице 2.6.

Таблица 2.6 — Ограничения БД и проверки приложения

Уровень	**Ограничения**
База данных	Уникальны логин, название категории, пары пользователь/мероприятие в member, мероприятие/автор в review и мероприятие/категория в event_category
База данных	Возраст больше нуля; оценка от 1 до 5; длина отзыва не более 2000 символов
Приложение	Создание и перенос мероприятия в прошлое запрещены; прошедшее мероприятие нельзя редактировать
Приложение	Изменение и удаление мероприятия доступны автору либо администратору
Приложение	Отзыв разрешен после завершения мероприятия, автор отзыва не является организатором; после удаления пробелов по краям текст содержит от 1 до 2000 символов

Удаление объявления изменяет его признак `deleted`; связанные записи сохраняются. При физическом удалении мероприятия каскадно удаляются участие, связи с категориями, отзывы и журнал. Физическое удаление пользователя каскадно удаляет его мероприятия, участия и отзывы. Удаление используемой локации, изображения или категории ограничивается внешними ключами.

2.3 Проверка соответствия третьей нормальной форме

Нормализация проверяется по функциональным зависимостям: одинаковые значения определяющих атрибутов должны задавать одинаковые значения зависимых атрибутов. Для третьей нормальной формы (3НФ) в каждой нетривиальной зависимости определяющий набор должен быть суперключом либо зависимый атрибут должен входить в потенциальный ключ [6].

Потенциальные ключи перечислены в таблице 2.7. Каждый из них определяет остальные атрибуты отношения.

Таблица 2.7 — Потенциальные ключи отношений

Отношение	**Потенциальные ключи**
users	id; login
category	id; name
member	id; (user_id, event_id)
review	id; (event_id, author_id)
event_category	(event_id, category_id)
event, location, image	id в каждом отношении
event_deletion_log	id в каждом отношении

Первая нормальная форма выполняется: каждый атрибут содержит одно значение выбранного домена. Участники мероприятия и его категории представлены отдельными строками отношений member и event_category, а не списками в одной ячейке. Содержимое изображения рассматривается как одно двоичное значение.

В отношениях users, event, location, image, category и event_deletion_log описательные атрибуты зависят от идентификатора записи. Например, для локации функциональная зависимость имеет вид

$$\mathrm{id} \to (\mathrm{name}, \mathrm{address}, \mathrm{longitude}, \mathrm{latitude}). \tag{2.1}$$

В users дополнительным потенциальным ключом является login, в category – name. Каждый из них также определяет все остальные атрибуты соответствующего отношения. Все потенциальные ключи перечисленных шести отношений состоят из одного атрибута, поэтому частичные зависимости от ключей отсутствуют.

В отношении member наряду с идентификатором используется потенциальный ключ из идентификаторов пользователя и мероприятия:

$$(\mathrm{user_id}, \mathrm{event_id}) \to \mathrm{role}. \tag{2.2}$$

Один пользователь может иметь разные роли в разных мероприятиях, а участники одного мероприятия – разные роли. Поэтому role зависит от всего составного ключа.

Аналогично для отношения review выполняется зависимость

$$(\text{event_id}, \text{author_id}) \rightarrow (\text{rating}, \text{text}, \text{created_at}). \tag{2.3}$$

Оценка, текст и время зависят от пары мероприятие/автор: один автор может оставить отзывы о разных мероприятиях, а мероприятие – получить отзывы разных авторов. В member и review составной ключ также определяет id, а id – остальные атрибуты.

В отношении event_category оба атрибута входят в составной первичный ключ, неключевых атрибутов нет. Таким образом, ни в одном из 9 отношений неключевые атрибуты не зависят от части какого-либо потенциального ключа. Условия второй нормальной формы выполнены.

Транзитивные зависимости исключены разделением описательных сведений: event содержит ссылки на организатора, локацию и изображение, а их характеристики хранятся в users, location и image. Аналогично названия категорий находятся только в category; связующие отношения содержат ссылки, журнал – идентификатор мероприятия и время удаления. Рейтинг организатора вычисляется по отзывам.

При проверке приняты следующие ограничения: названия локаций и их координаты не считаются уникальными; город мероприятия задается отдельно от адреса локации, поэтому зависимость location_id $\rightarrow$ city в event не задана. В журнале одному мероприятию могут соответствовать несколько фактов удаления, поэтому event_id не определяет deleted_at.

При принятом наборе функциональных зависимостей неключевые атрибуты определяются потенциальными ключами своих отношений и не зависят транзитивно от них через другие неключевые атрибуты. Следовательно, все 9 отношений разработанной базы данных соответствуют 3НФ.

2.4 Ролевая модель

Приложение поддерживает три подключения к БД, каждое со своей ролью: `joinup_guest`, `joinup_user` и `joinup_admin`. Для обработки запроса выбирается подключение, соответствующее роли пользователя: гость, пользователь или администратор. Матрица доступа приведена в таблице 2.8.

Таблица 2.8 — Права ролей БД

Объекты	**Гость**	**Пользователь**	**Администратор**
event, location, image, category, event_category, review	Чтение	Чтение, добавление, изменение, удаление	Все права на таблицы
users, member, event_deletion_log	Нет	Чтение, добавление, изменение, удаление	Все права на таблицы
Последовательности идентификаторов восьми таблиц с полем id	Нет	Использование и чтение	Все права на последовательности
Подключение к БД, использование схемы public	Да	Да	Да

Полные права администратора относятся к указанным объектам, а не к управлению сервером БД. Принадлежность мероприятия автору и полномочия администратора приложения проверяются перед вызовом репозитория.

Пользовательская роль БД также имеет право добавления в журнал, необходимое триггерной функции. Журнал не защищен от прямого изменения этой ролью. Роль гостя БД имеет доступ к разрешенным таблицам целиком; фильтрация удаленных мероприятий выполняется приложением.

2.5 Используемый триггер

Триггер `event_deletion_log_trigger` вызывает функцию `log_event_deletion` после обновления признака `deleted` для каждой измененной строки. Вход функции – прежнее и новое состояния строки; результат – запись журнала при переходе признака из ложного значения в истинное.

Схема алгоритма представлена на рисунке 2.2. Повторное присваивание истинного значения не создает запись. Возврат признака в ложное значение также не записывается. При ошибке вставки исходное обновление откатывается вместе с действиями триггера. Журнал содержит идентификатор мероприятия и время удаления, без копии его названия и описания.

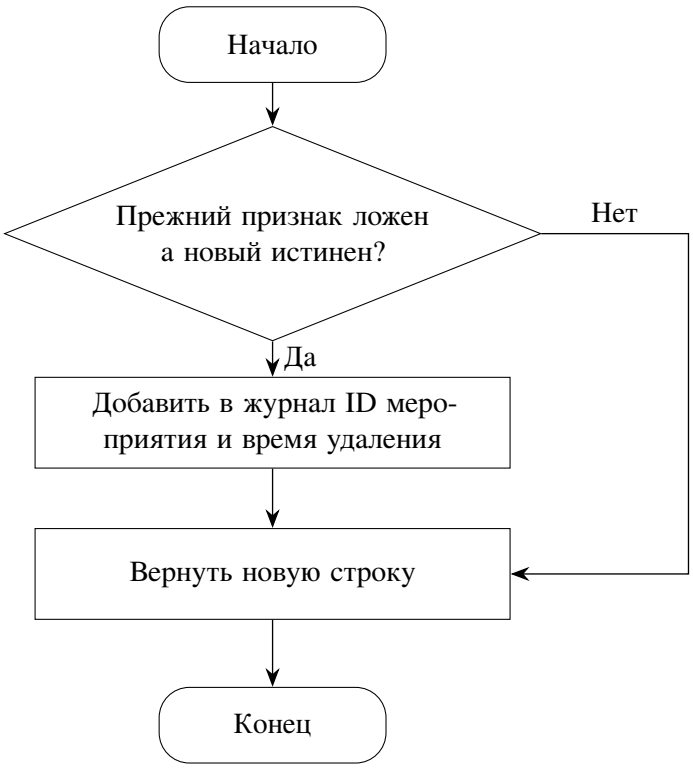


Рисунок 2.2 — Схема алгоритма функции записи факта удаления

2.6 Архитектура приложения

Приложение состоит из frontend-части, backend-части и БД. Схема представлена на рисунке 2.3. Контроллеры принимают запросы и формируют ответы, сервисы проверяют правила предметной области, репозитории выполняют запросы к БД. Для выбора и отображения места frontend обращается к API Яндекс Карт [7].

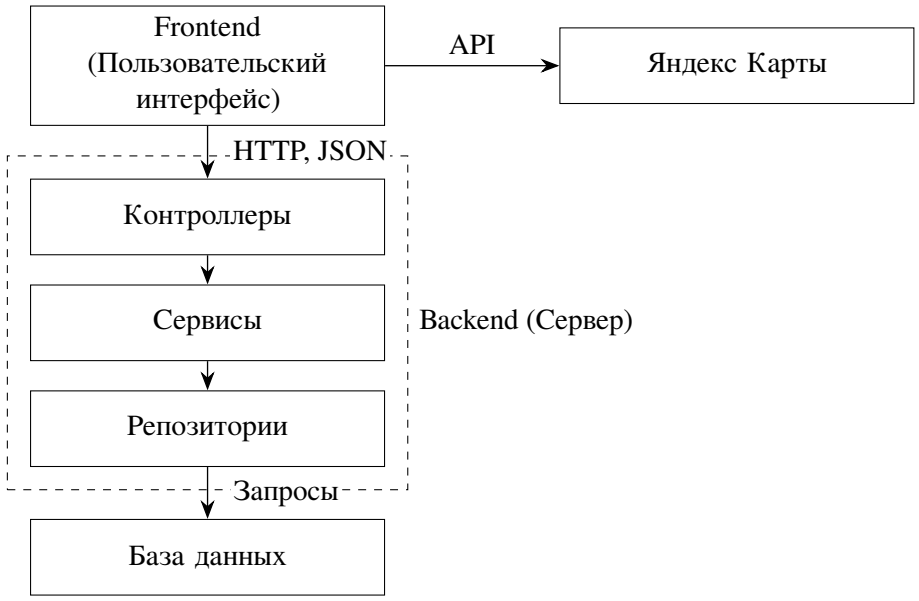


Рисунок 2.3 — Архитектура приложения

Сервер использует параметризованные запросы. Объект управления транзакцией передается репозиториям через контекст операции. Границы транзакций приведены в таблице 2.9.

Таблица 2.9 — Транзакции изменения данных

Операция	**Действия в одной транзакции**
Создание мероприятия	Проверка ссылок, добавление локации и мероприятия
Загрузка обложки	Добавление изображения и изменение ссылки на него в мероприятии
Создание отзыва	Чтение мероприятия, проверка условий и отсутствия отзыва, добавление отзыва
Мягкое удаление	Обновление признака удаления и вставка в журнал триггером в рамках одного оператора

Уникальные ограничения предотвращают повторное участие и повторный отзыв при одновременных запросах. Изменения локации и мероприятия выполняются в разных транзакциях.

2.7 Вывод

Определены структура 9 таблиц, ограничения и права ролей; для принятых зависимостей показано соответствие 3НФ. Разработаны схема триггера, архитектура приложения и границы транзакций.

3 Технологическая часть

3.1 Выбор системы управления базами данных

Для реализации рассмотрены PostgreSQL, MySQL и SQLite. PostgreSQL и MySQL поддерживают внешние ключи, транзакции, построчные триггеры и права на таблицы [8—11]. В SQLite есть триггеры и транзакции [12; 13], но внешние ключи требуют включения, а разграничение прав командами `GRANT` и `REVOKE` отсутствует [14; 15]. Поэтому SQLite не подходит для требуемой ролевой модели.

Выбрана PostgreSQL 16: она поддерживает необходимые ограничения, триггер и роли БД, а драйвер pgx обеспечивает доступ к ней из языка Golang.

3.2 Выбор средств реализации приложения

Сервер написан на Go 1.24. Для маршрутизации HTTP-запросов используется Echo 5, для доступа к PostgreSQL – pgx/v5, для применения SQL-миграций – Goose 3. Клиентская часть написана на React 18 и TypeScript; сборку выполняет Vite.

HTTP-запросы передаются сервисам и репозиториям по схеме на рисунке 2.3. Авторизация выполняется по токену пользователя.

3.3 Структуры приложения и базы данных

Команда `make migrate_up` применяет SQL-миграции через Goose. Сценарий создания таблиц, ограничений, триггера, индекса и ролей приведен в приложении А.

Идентификаторы созданы типом `serial`, у review и event_deletion_log – `bigserial`. Строковые данные хранятся в `text`, время – в `timestamp`, признак удаления – в `boolean`, изображения – в `bytea`, координаты – в `float`. Ограничения из таблицы 2.6 реализованы первичными и внешними ключами, `UNIQUE`, уникальными индексами и `CHECK`; проверки приложения выполняются в сервисах.

Для поиска создан частичный индекс `event_search_city_date_idx` по полям `city`, `event_date`, `id` при `deleted = false`. Он соответствует фильтрации по городу и диапазону времени; исследование его влияния приведено в исследовательской части.

Триггер вызывает `log_event_deletion` после обновления поля `deleted`; алгоритм приведен на рисунке 2.2. Миграция ролей назначает права на таблицы и последовательности по матрице из таблицы 2.8. Код функции, триггера и выдачи прав включен в приложение А.

Репозитории выполняют запросы через выбранный пул соединений `pgxpool.Pool`. Значения фильтров и изменяемых полей передаются параметрами `$1`, `$2` и т. д. Границы транзакций приведены в таблице 2.9.

Поиск выполняется методом `SearchEvents`. Запрос исключает удаленные объявления, соединяет `event` с `location`, применяет фильтры названия, дат и города. Для выбора мероприятий, содержащих все указанные категории, применяется подзапрос с группировкой по мероприятию и условием `HAVING COUNT(DISTINCT category_id)`. Результаты упорядочиваются по времени

и идентификатору. После основного запроса сведения об авторах, участниках и категориях загружаются отдельными запросами.

Простой запрос читает пользователя по логину. В `GetUser` агрегатная функция `AVG` вычисляет рейтинг организатора, подзапрос подсчитывает отзывы. Изменяющие запросы добавляют мероприятие и обновляют признак удаления.

3.4 Интерфейс приложения

В клиентском приложении имеются экраны регистрации, входа, поиска, созданных мероприятий, участий, профиля и карточки мероприятия. Поиск с заданными фильтрами показан на рисунке 3.1. В карточке мероприятия (рисунок 3.2) видны описание, место проведения, карта и ссылка на чат. Пользователь может присоединиться к мероприятию, автор – изменить или удалить его. Изображения загружаются через сервер; для выбора адреса и отображения карты используются API Яндекс Карт.

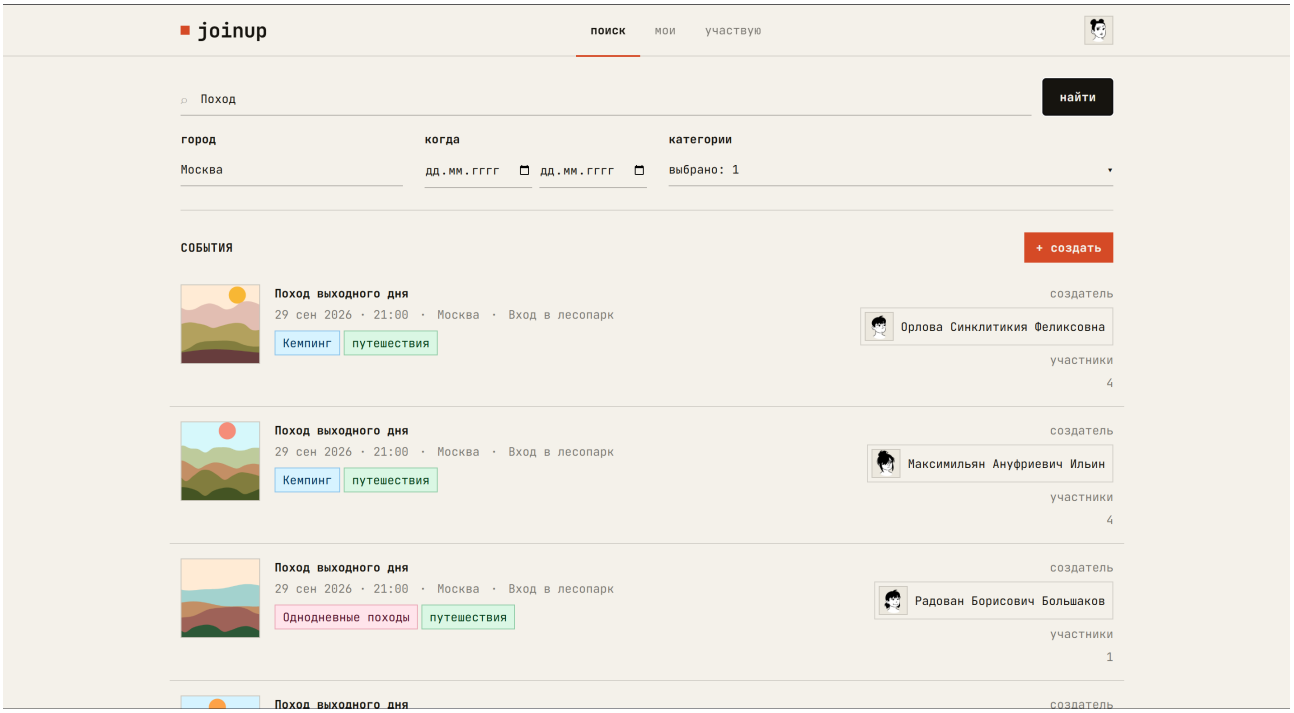


Рисунок 3.1 — Поиск мероприятий с фильтрами

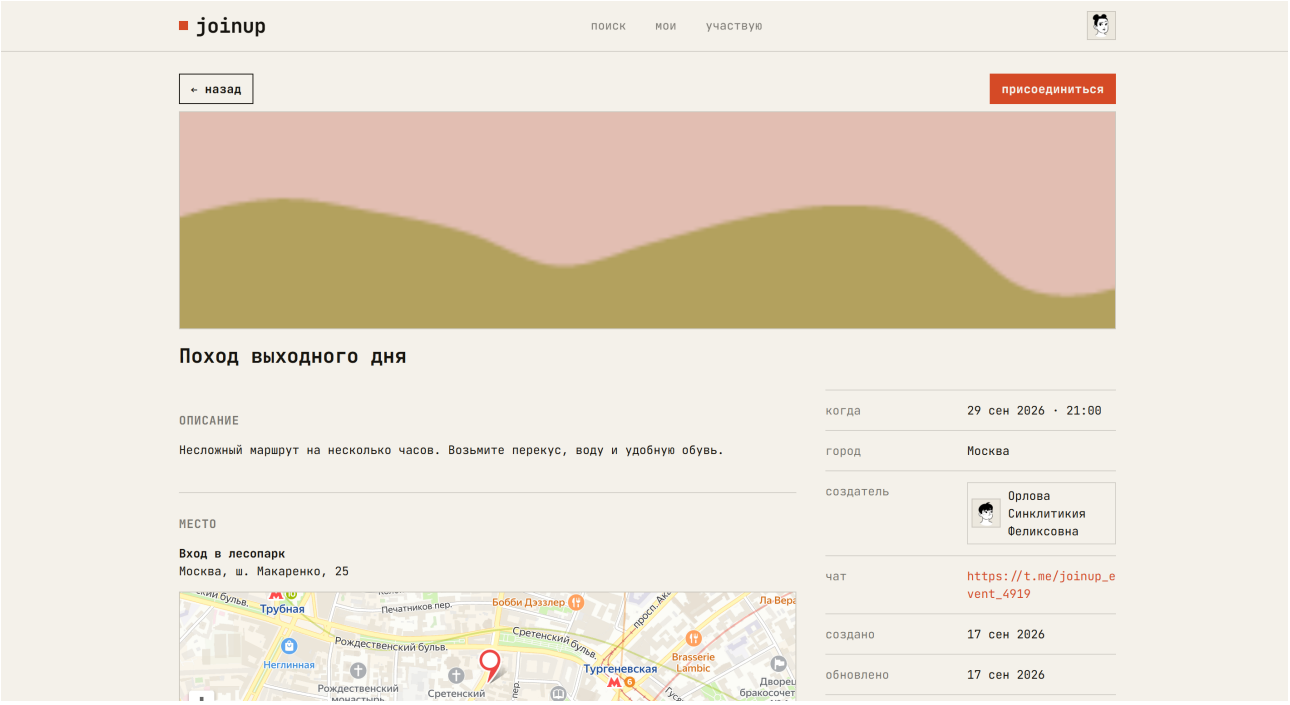


Рисунок 3.2 — Карточка мероприятия

При редактировании автор изменяет название, описание, время и место проведения мероприятия. Профиль с показателями участия и рейтингом представлен на рисунке 3.3.

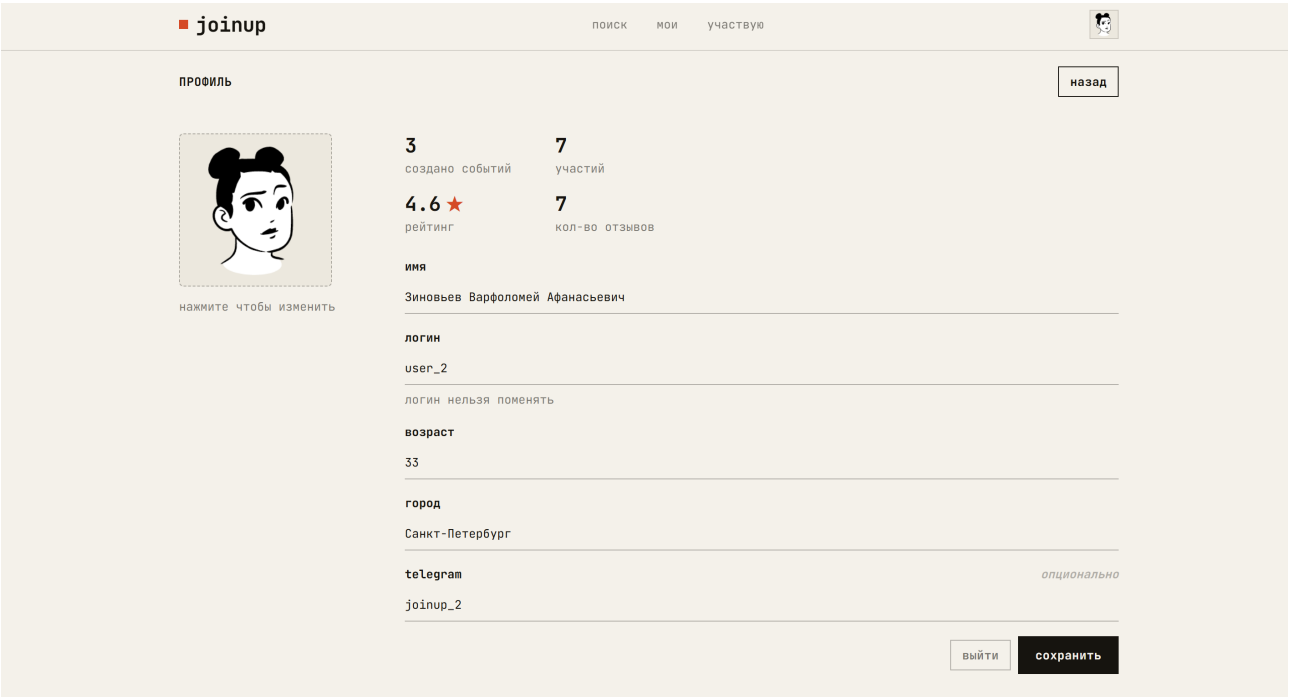


Рисунок 3.3 — Профиль пользователя

Учебные данные созданы сценарием app/scripts/seed.py: пользователи, мероприятия, участия и отзывы. Некоторые мероприятия помечены удаленными. Количества строк приведены в таблице 3.1.

Таблица 3.1 — Количество строк в таблицах БД

Таблица	**Строк**	**Таблица**	**Строк**
users	5001	event	10113
location	10113	image	15114
member	32406	category	99
event_category	20224	review	12853
event_deletion_log	505		

3.5 Тестирование

Модульные тесты проверяют бизнес-логику сервисов с подменой зависимостей; интеграционные – работу HTTP-обработчиков и репозиториев с настоящей PostgreSQL. Результаты сравниваются с ожидаемыми ответами и состоянием БД. Модульные тесты выполнены командой `go test ./internal/service`; они завершились успешно.

Четыре интеграционных сценария были выполнены в тестовом контейнере PostgreSQL 16 с миграциями проекта. Перед каждым сценарием таблицы очищались и заполнялись тестовыми данными. Для имитации API-вызовов использовался стандартный пакет httptest [16]; сохраненные мероприятие и локация проверялись SQL-запросом.

Негативные сценарии напрямую проверяли ограничения БД: повтор логина, ссылку на отсутствующую локацию и оценку отзыва вне диапазона. Сверялись код SQLSTATE, имя ограничения и состояние данных после отказа. Результаты приведены в таблице 3.2.

Таблица 3.2 — Проверка создания мероприятия и ограничений БД

Сценарий	**Ожидалось**	**Получено**
Создание мероприятия с новой локацией	HTTP 200; данные мероприятия и локации сохранены	HTTP 200; прочитанные значения совпали с заданными
Повтор логина admin	SQLSTATE 23505; второй пользователь не добавлен	SQLSTATE 23505; в users одна строка
Замена локации на отсутствующую (ID 9999)	SQLSTATE 23503; прежняя связь сохранена	SQLSTATE 23503; мероприятие связано с существующей локацией
Добавление отзыва с оценкой 0	SQLSTATE 23514; отзыв не добавлен	SQLSTATE 23514; в review нет строк

При ручной HTTP-проверке вход с верным паролем возвращал HTTP 200, с неверным – HTTP 401; поиск без токена – HTTP 401. Поиск по названию «Разговорный клуб английского» и городу Москва вернул HTTP 200 и только неудаленные мероприятия.

Права БД проверены отдельными подключениями: гостю разрешено чтение event и запрещены ее изменение и чтение users; пользователю разрешены чтение и изменение event; администратору – удаление из event. Все шесть результатов совпали с ожидаемыми.

Для функции `log_event_deletion` выделены классы эквивалентности в таблице 3.3. Проверка проведена на мероприятии с `id=1` внутри транзакции с последующим откатом. Указано изменение числа строк журнала для этого мероприятия.

Таблица 3.3 — Классы входных данных триггерной функции

Изменение признака	**Ожидалось**	**Получено**
`false -> true`	+1	+1
`false -> false`	0	0
`true -> true`	0	0
`true -> false`	0	0

После отката транзакции признак удаления и число строк журнала вернулись к исходным значениям.

3.6 Вывод

Реализованы БД PostgreSQL, сервер Go и клиент React. Описаны методы тестирования; проверены создание мероприятия, ограничения БД, права ролей и триггер удаления. Ожидаемые и фактические результаты совпали.

4 Исследовательская часть

4.1 Технические характеристики

Технические характеристики компьютера, на котором проводились измерения:

— операционная система – Windows 11;

— оперативная память – 32 ГБ;

— процессор – AMD Ryzen 7 7700.

PostgreSQL 16.11 работала в WSL 2; для выполнения запросов использовалась библиотека psycopg 3.3.6 на языке Python версии 3.12.3. Данные размещены во временных таблицах. Кеш БД перед измерениями прогревался и не сбрасывался. Все неиспользуемые для измерений приложения на компьютере были закрыты. Антивирус отключен.

4.2 Постановка исследования

Цель исследования – определить, как меняется время запроса поиска при наличии индекса по городу и дате и без него для 1000, 2000, 5000 и 10000 мероприятий. Использован запрос метода `SearchEvents`, который соединяет мероприятие с локацией, исключает удаленные объявления, применяет фильтры и сортирует результат по дате и идентификатору. Загрузка категорий, участников и авторов после основного запроса не измерялась.

В обоих вариантах заданы город Москва и интервал от 26.08.2026 до 25.10.2026; фильтры названия и категорий не заданы. Число локаций постоянно и равно 12006; около 5% мероприятий помечены удаленными.

Определение исследуемого индекса приведено в листинге 4.1.

Листинг 4.1 — Индекс для исследования поиска

```
create index research_city_date_idx
    on event (city, event_date, id)
    where deleted = false;
```

Для каждой комбинации объема данных и наличия индекса проведено 10 блоков по 50 измерений. Перед каждым блоком выполнено 10 прогревочных запросов. Порядок вариантов с индексом и без него чередовался между блоками. Таким образом, на каждую точку графика приходится 500 измерений, всего проведено 4000 измерений. Все значения включены в расчет среднего.

Измеряемая величина – серверное время Execution Time команды `EXPLAIN (ANALYZE, BUFFERS, TIMING OFF, FORMAT JSON)`. Время планирования записано отдельно и в результат не входит. Среднее вычислено по 500 значениям. Погрешность на графике и в таблице представляет половину ширины 95%-го доверительного интервала среднего времени.

При неизвестном стандартном отклонении генеральной совокупности граница погрешности среднего рассчитывается по формуле:

$$\Delta = t_{1-\alpha/2;\,n-1}\frac{s}{\sqrt{n}}, \tag{4.1}$$

где n – число наблюдений, s – их выборочное стандартное отклонение, α – уровень значимости, $t_{1-\alpha/2;\,n-1}$ – квантиль распределения Стьюдента с $n-1$ степенями свободы. Доверительный интервал имеет границы $\bar{x}-\Delta$ и $\bar{x}+\Delta$, где $\bar{x}$ – выборочное среднее.

В исследовании наблюдениями для расчета погрешности служат средние времена десяти блоков, поэтому $n = 10$, а s характеризует разброс этих средних и выражается в миллисекундах. Для доверительной вероятности 0,95 уровень значимости $\alpha = 0{,}05$; при $n-1 = 9$ степенях свободы квантиль $t_{0{,}975;\,9} \approx 2{,}262$. Подстановка в формулу (4.1) дает

$$\Delta = 2{,}262\frac{s}{\sqrt{10}}. \tag{4.2}$$

Расчет предполагает независимость и приближенно нормальное распределение средних блоков. Из-за возможной корреляции блоков интервал используется только для ориентировочного сравнения.

4.3 Определение зависимости времени поиска от количества мероприятий

Средние значения замеров приведены в таблице 4.1.

Таблица 4.1 — Время выполнения запроса поиска

Записей	**Без индекса, мс**	**С индексом, мс**	**Найдено**
1000	$0{,}325 \pm 0{,}005$	$0{,}199 \pm 0{,}004$	57
2000	$1{,}824 \pm 0{,}054$	$1{,}583 \pm 0{,}077$	105
5000	$2{,}673 \pm 0{,}080$	$1{,}973 \pm 0{,}043$	287
10000	$4{,}905 \pm 0{,}320$	$3{,}038 \pm 0{,}203$	600

Зависимость времени от числа мероприятий представлена на рисунке 4.1.

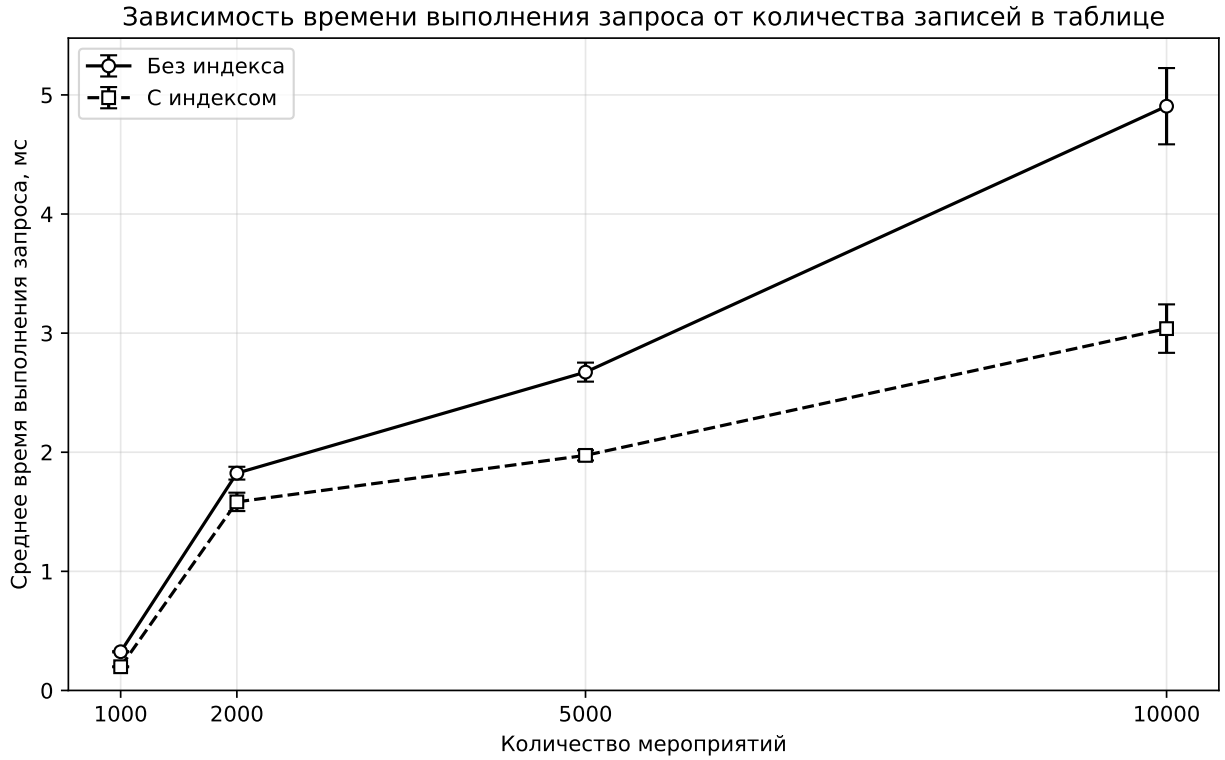


Рисунок 4.1 — Зависимость времени поиска от числа мероприятий

В сохраненных планах запрос без дополнительного индекса сканирует таблицу `event` последовательно. При наличии индекса применяется Bitmap Index Scan с последующим Bitmap Heap Scan. Для 1000 мероприятий соединение с таблицей `location` выполнено вложенным циклом, для остальных объемов – хеш-соединением. Смена способа соединения влияет на рост времени между 1000 и 2000 строками.

4.4 Вывод

Во всех четырех наборах среднее время с индексом меньше, чем без него. Для 10000 мероприятий оно составило 3,038 мс вместо 4,905 мс, то есть сократилось примерно на 38%. Результат относится к одному SQL-запросу, заданным фильтрам, тестовым данным и прогретому кешу временных таблиц. Он не характеризует время загрузки страницы или работу всего приложения под нагрузкой, а показывает влияние индекса на скорость выполнения основного поискового запроса внутри приложения.

ЗАКЛЮЧЕНИЕ

Цель работы достигнута: спроектированы и реализованы база данных для поиска и организации досуга и приложение для взаимодействия с ней. Для достижения поставленной цели были выполнены следующие задачи:

1) проанализированы предметная область и известные решения;
2) определены требования, сущности и связи между ними;
3) спроектированы БД, ограничения целостности, роли и триггер сохранения истории удаления объявлений, обосновано соответствие БД третьей нормальной форме;
4) выбраны средства реализации и разработано приложение JoinUp;
5) описаны методы тестирования, проверены создание мероприятия, три ограничения БД, права ролей и работа триггера; результаты совпали с ожидаемыми;
6) исследовано время поиска мероприятий с индексом по основным фильтрам и без него.

В результате исследования установлено, что во всех рассмотренных наборах данных индекс уменьшает среднее время выполнения основного запроса поиска. При 10000 мероприятий время составило 4,905 мс без индекса и 3,038 мс с ним, то есть сократилось примерно на 38%. Полученный результат относится к условиям проведенного исследования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Meetup [Электронный ресурс]. — URL: https://www.meetup.com/ (дата обращения: 08.10.2026).
2. Mobilizon [Электронный ресурс]. — URL: https://mobilizon.org/ (дата обращения: 08.10.2026).
3. Timepad [Электронный ресурс]. — URL: https://timepad.ru/ (дата обращения: 08.10.2026).
4. What is data modeling? – IBM [Электронный ресурс]. — URL: https://www.ibm.com/think/topics/data-modeling (дата обращения: 24.09.2026).
5. Data Modeling in MongoDB – Database Manual [Электронный ресурс]. — URL: https://www.mongodb.com/docs/manual/data-modeling/ (дата обращения: 24.09.2026).
6. Normalization: From Anomalies to Normal Forms – Aalto OpenCS [Электронный ресурс]. — URL: https://opencs.aalto.fi/en/courses/databases/part-8/4-normalization (дата обращения: 25.09.2026).
7. JavaScript API 2.1 – Яндекс Карты [Электронный ресурс]. — URL: https://yandex.ru/dev/jsapi-v2-1/doc/ru/ (дата обращения: 08.10.2026).
8. Supported Features – PostgreSQL 16 Documentation [Электронный ресурс]. — URL: https://www.postgresql.org/docs/16/features-sql-standard.html (дата обращения: 25.09.2026).
9. Privileges – PostgreSQL 16 Documentation [Электронный ресурс]. — URL: https://www.postgresql.org/docs/16/ddl-priv.html (дата обращения: 25.09.2026).
10. Privileges Provided by MySQL – MySQL 8.4 Reference Manual [Электронный ресурс]. — URL: https://dev.mysql.com/doc/refman/8.4/en/privileges-provided.html (дата обращения: 25.09.2026).
11. InnoDB Transaction Model – MySQL 8.4 Reference Manual [Электронный ресурс]. — URL: https://dev.mysql.com/doc/refman/8.4/en/innodb-transaction-model.html (дата обращения: 25.09.2026).
12. CREATE TRIGGER – SQLite [Электронный ресурс]. — URL: https://www.sqlite.org/lang_createtrigger.html (дата обращения: 25.09.2026).
13. Transaction – SQLite [Электронный ресурс]. — URL: https://www.sqlite.org/lang_transaction.html (дата обращения: 25.09.2026).
14. SQLite Foreign Key Support – SQLite [Электронный ресурс]. — URL: https://www.sqlite.org/foreignkeys.html (дата обращения: 25.09.2026).
15. SQL Features That SQLite Does Not Implement – SQLite [Электронный ресурс]. — URL: https://www.sqlite.org/omitted.html (дата обращения: 25.09.2026).
16. httptest – Go Packages [Электронный ресурс]. — URL: https://pkg.go.dev/net/http/httptest (дата обращения: 08.10.2026).

ПРИЛОЖЕНИЕ А

Команды создания объектов текущей схемы PostgreSQL приведены в листингах А.1–А.7 в порядке их применения.

Листинг А.1 — Основные таблицы

```
create table if not exists image (
    id serial primary key,
    name text not null,
    data bytea not null
);

create table if not exists location (
    id serial primary key,
    name text not null,
    longitude float not null,
    latitude float not null,
    address text not null
);

create table if not exists users (
    id serial primary key,
    name text not null,
    age int not null check (age > 0),
    login text not null,
    password text not null,
    created_at timestamp not null,
    city text not null,
    telegram_login text,
    avatar_id int references image(id)
);

create table if not exists event (
    id serial primary key,
    name text not null,
    description text,
    created_at timestamp not null,
    updated_at timestamp not null,
    event_date timestamp not null,
    telegram_chat_url text,
    city text not null,
    creator_id int not null references users(id) on delete cascade,
    location_id int not null references location(id),
    image_id int references image(id),
    deleted boolean not null default false
);

create table if not exists member (
```

```
    id serial primary key,
    user_id int not null references users(id) on delete cascade,
    event_id int not null references event(id) on delete cascade,
    role text not null
);

create table if not exists category (
    id serial primary key,
    subcategory_id int references category(id),
    name text not null constraint category_name_unique unique
);

create table if not exists event_category (
    event_id int not null references event(id) on delete cascade,
    category_id int not null references category(id),
    primary key (event_id, category_id)
);
```

Листинг А.2 — Ограничение логина и роль пользователя

```
alter table users
    add constraint users_login_unique unique (login),
    add column role text not null default 'user';
```

Листинг А.3 — Категории и уникальность участия

```
create unique index if not exists member_user_event_unique
    on member (user_id, event_id);

insert into category (name)
values
    ('спорт'),
    ('музыка'),
    ('игры'),
    ('арт'),
    ('прогулки'),
    ('учёба'),
    ('еда'),
    ('tech'),
    ('путешествия'),
    ('фото'),
    ('кино')
on conflict (name) do nothing;
```

Листинг А.4 — Отзывы

```
create table if not exists review (
    id          bigserial primary key,
    event_id    bigint    not null
        references event(id) on delete cascade,
```

```
    author_id   bigint    not null
        references users(id) on delete cascade,
    rating      integer   not null
        constraint review_rating_check check (rating between 1 and 5),
    text        text      not null
        constraint review_text_length_check check (char_length(text) <= 2000),
    created_at  timestamp not null default (now() at time zone 'utc'),
    constraint review_event_author_unique unique (event_id, author_id)
);

create index if not exists review_event_id_idx on review (event_id);
create index if not exists review_event_creator_lookup_idx on review (event_id);
```

Листинг A.5 — Журнал и триггер удаления

```
create table if not exists event_deletion_log (
    id bigserial primary key,
    event_id int not null references event(id) on delete cascade,
    deleted_at timestamp not null default (now() at time zone 'utc')
);

-- +goose StatementBegin
create or replace function log_event_deletion()
returns trigger
language plpgsql
as $$
begin
    if old.deleted = false and new.deleted = true then
        insert into event_deletion_log (event_id)
        values (new.id);
    end if;

    return new;
end;
$$;
-- +goose StatementEnd

create trigger event_deletion_log_trigger
after update of deleted on event
for each row
execute function log_event_deletion();
```

Листинг A.6 — Индекс поиска

```
create index if not exists event_search_city_date_idx
    on event (city, event_date, id)
    where deleted = false;
```

Листинг A.7 — Роли и права доступа

```
-- +goose StatementBegin
do $$
begin
    if not exists (select 1 from pg_roles where rolname = 'joinup_guest') then
        create role joinup_guest login password 'guest';
    end if;
    if not exists (select 1 from pg_roles where rolname = 'joinup_user') then
        create role joinup_user login password 'user';
    end if;
    if not exists (select 1 from pg_roles where rolname = 'joinup_admin') then
        create role joinup_admin login password 'admin';
    end if;
    execute format('grant connect on database %I to joinup_guest, joinup_user,
    joinup_admin', current_database());
end;
$$;
-- +goose StatementEnd

grant usage on schema public to joinup_guest, joinup_user, joinup_admin;

revoke all privileges on all tables in schema public from joinup_guest, joinup_user,
    joinup_admin;
revoke all privileges on all sequences in schema public from joinup_guest, joinup_user,
    joinup_admin;

grant select on event, location, image, category, event_category, review to joinup_guest;

grant select, insert, update, delete on
    image,
    location,
    users,
    event,
    member,
    category,
    event_category,
    review,
    event_deletion_log
to joinup_user;

grant usage, select on
    image_id_seq,
    location_id_seq,
    users_id_seq,
    event_id_seq,
    member_id_seq,
    category_id_seq,
    review_id_seq,
    event_deletion_log_id_seq
to joinup_user;
```

```
grant all privileges on
    image,
    location,
    users,
    event,
    member,
    category,
    event_category,
    review,
    event_deletion_log
to joinup_admin;

grant all privileges on
    image_id_seq,
    location_id_seq,
    users_id_seq,
    event_id_seq,
    member_id_seq,
    category_id_seq,
    review_id_seq,
    event_deletion_log_id_seq
to joinup_admin;
```